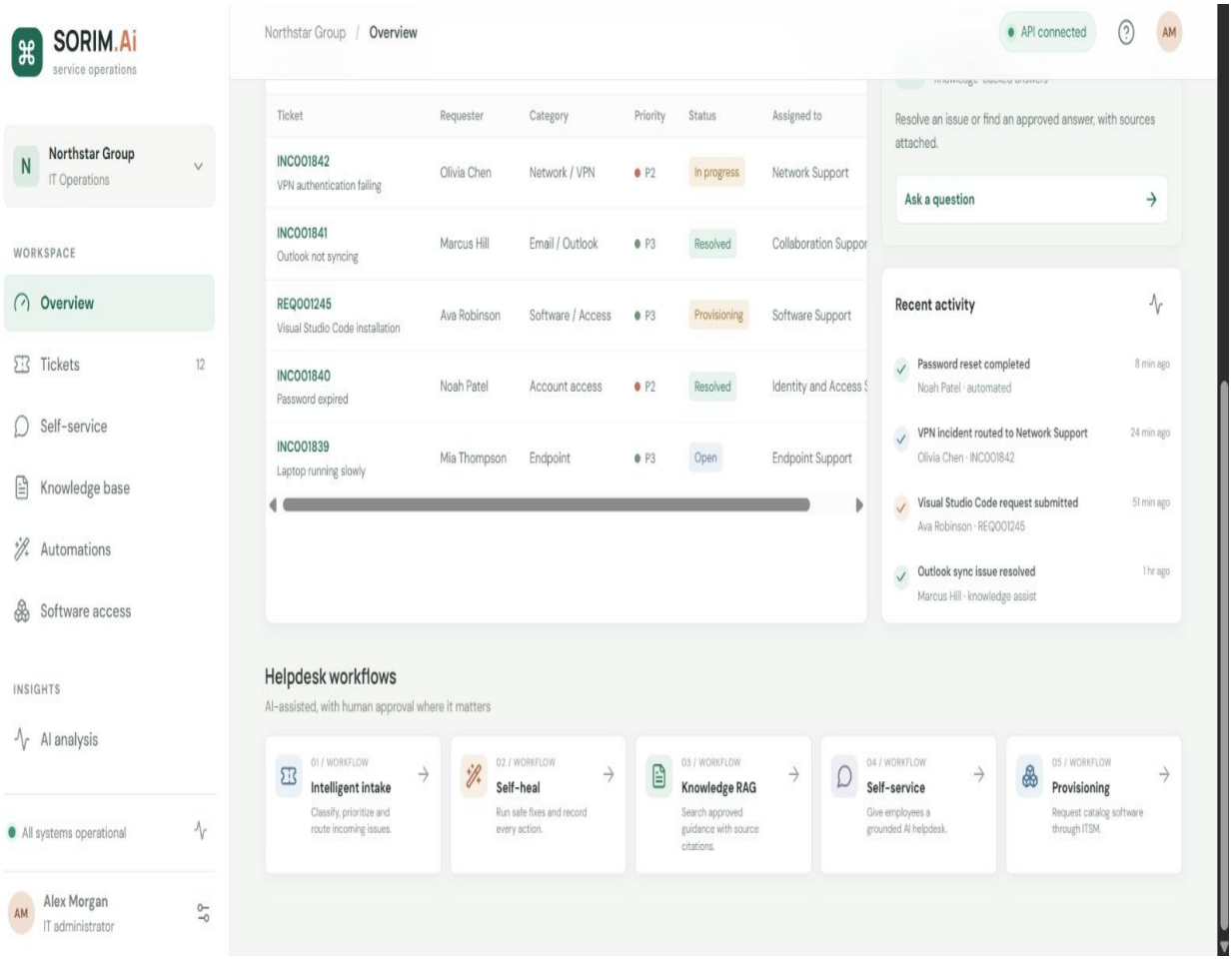
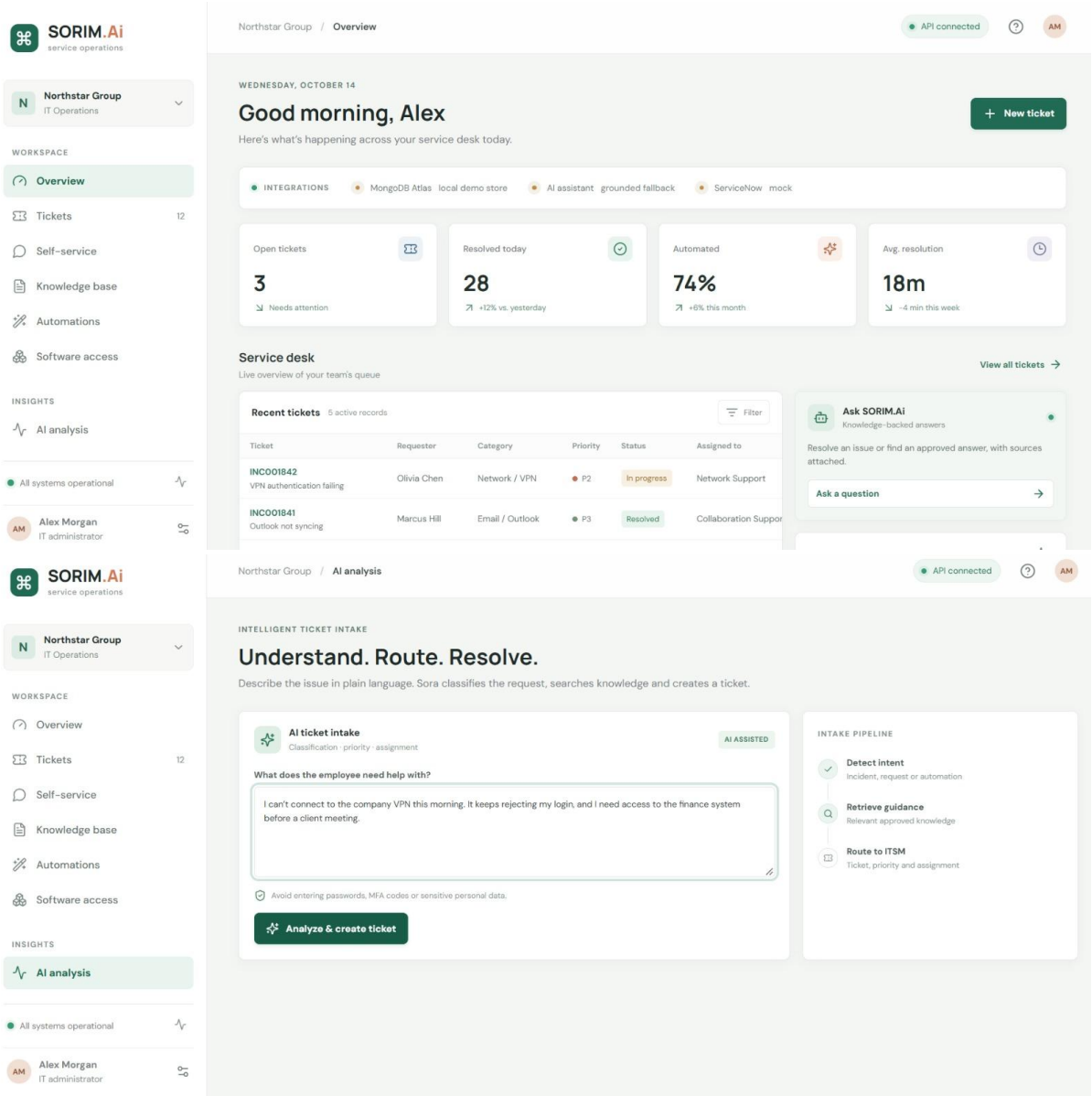
AI POWERED ITSM & INTELLIGENT HLPDESK AUTOMATION PLATFORM





SORIM.Ai ITSM Helpdesk API

1.0.0 OAS 3.1

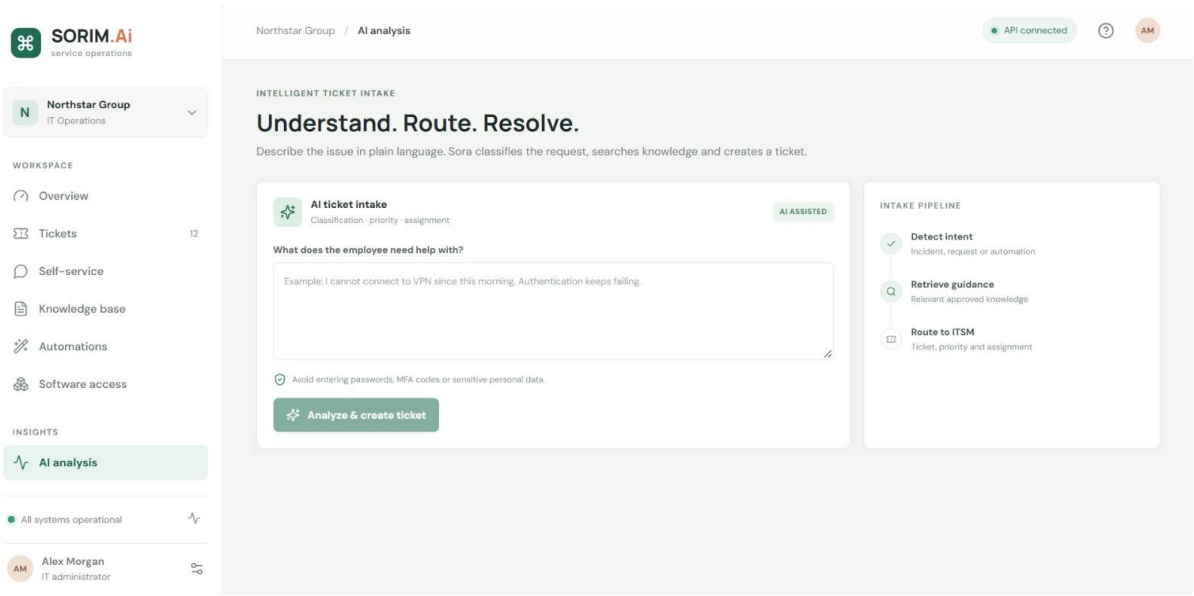
/openapi.json

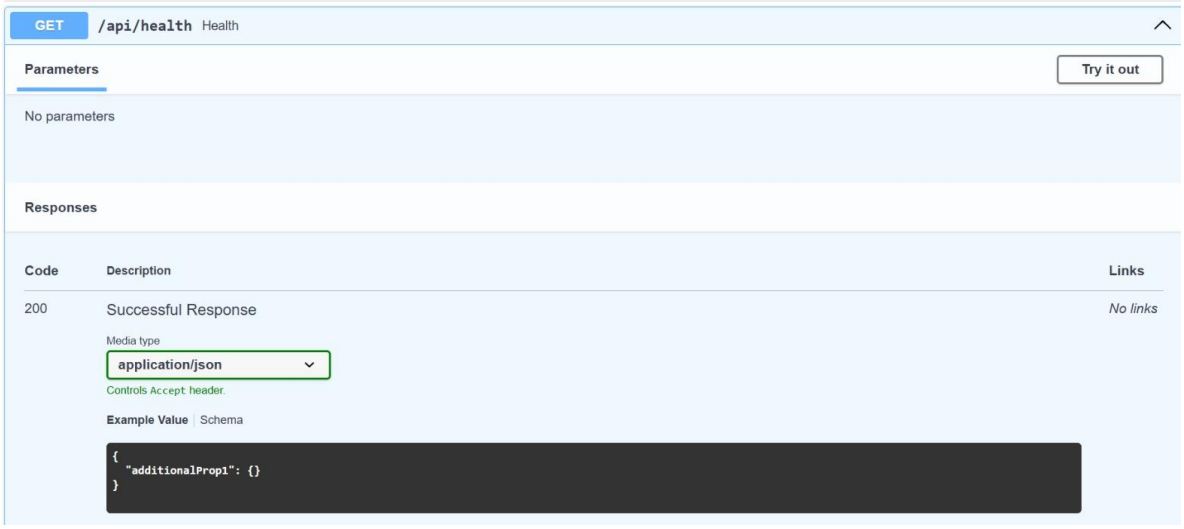
default

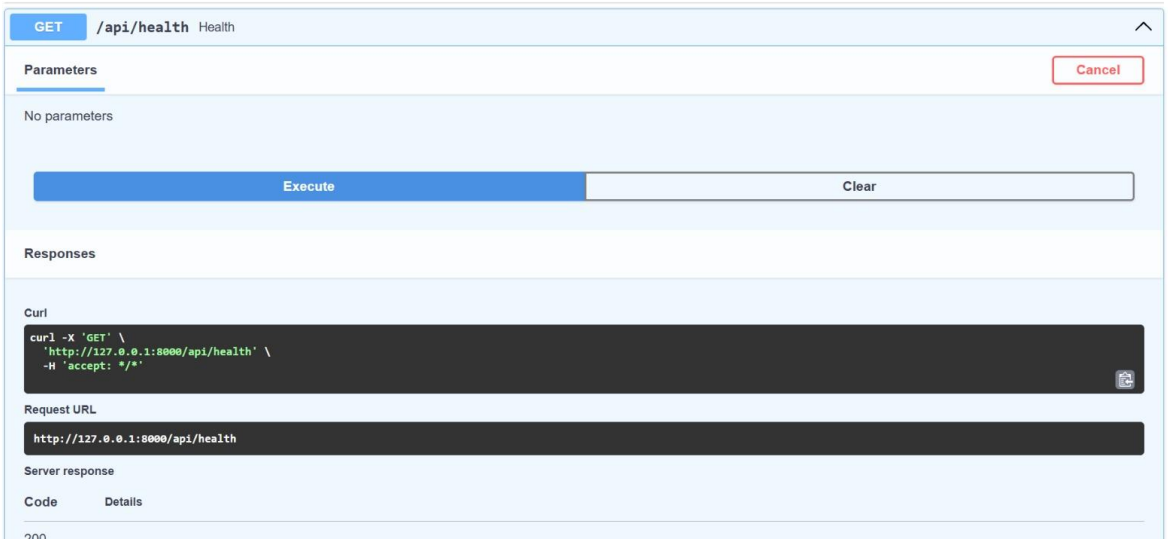
GET	/api/health	Health
GET	/api/dashboard	Dashboard
GET	/api/tickets	Tickets
GET	/api/knowledge	Knowledge
POST	/api/intake	Intake
POST	/api/chat	Chat
GET	/api/servicenow/incidents/{incident_number}	Get Incident
PATCH	/api/servicenow/incidents/{incident_number}	Update Incident

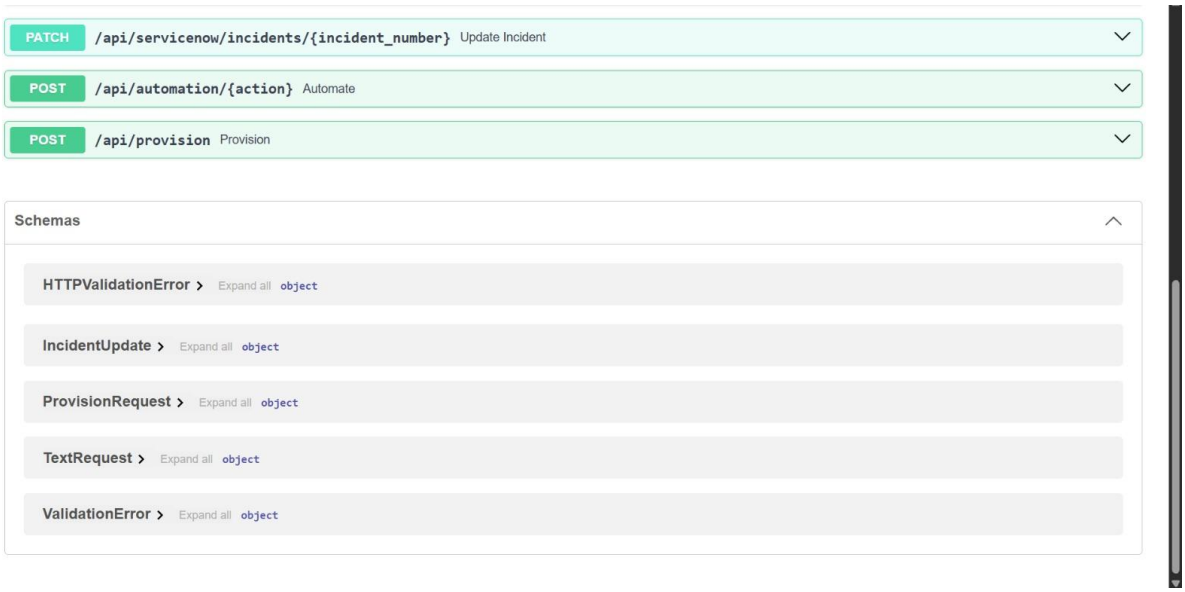
default

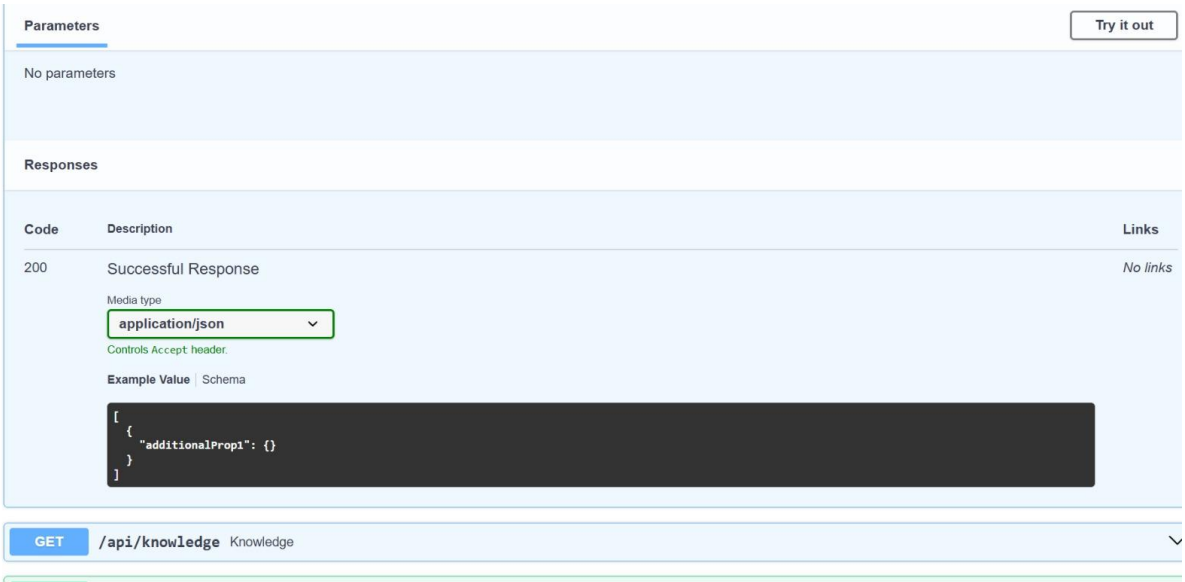
GET	`/api/health`	Health
GET	`/api/dashboard`	Dashboard
GET	`/api/tickets`	Tickets
GET	`/api/knowledge`	Knowledge
POST	`/api/intake`	Intake
POST	`/api/chat`	Chat
GET	`/api/servicenow/incidents/{incident_number}`	Get Incident
PATCH	`/api/servicenow/incidents/{incident_number}`	Update Incident
POST	`/api/automation/{action}`	Automate
POST	`/api/provision`	Provision






GET /api/health Health
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/api/health' \
-H 'accept: */*'
Request URL
http://127.0.0.1:8000/api/health
Server response
Code
Details


PATCH /api/servicenow/incidents/{incident_number} Update Incident
POST /api/automation/{action} Automate
POST /api/provision Provision
Schemas
HTTPValidationError Expand all object
IncidentUpdate Expand all object
ProvisionRequest Expand all object
TextRequest Expand all object
ValidationError Expand all object


Parameters
Try it out
No parameters
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
[
{
"additionalProp1": {}
}
]

GET /api/knowledge Knowledge

Schemas

HTTPValidationError ^ Collapse all object

detail > Expand all array<object>

IncidentUpdate ^ Collapse all object

status > Expand all (string | null)

assigned_to > Expand all (string | null)

title > Expand all (string | null)

ProvisionRequest ^ Collapse all object

software* string [2, 120] characters

requester > Expand all string

justification > Expand all string

TextRequest ^ Collapse all object

message* string [3, 2000] characters

requester > Expand all string

ValidationError > Expand all object

status > Expand all (string | null)

assigned_to > Expand all (string | null)

title > Expand all (string | null)

ProvisionRequest ^ Collapse all object

software* string [2, 120] characters

requester > Expand all string

justification > Expand all string

TextRequest ^ Collapse all object

message* string [3, 2000] characters

requester > Expand all string

ValidationError ^ Collapse all object

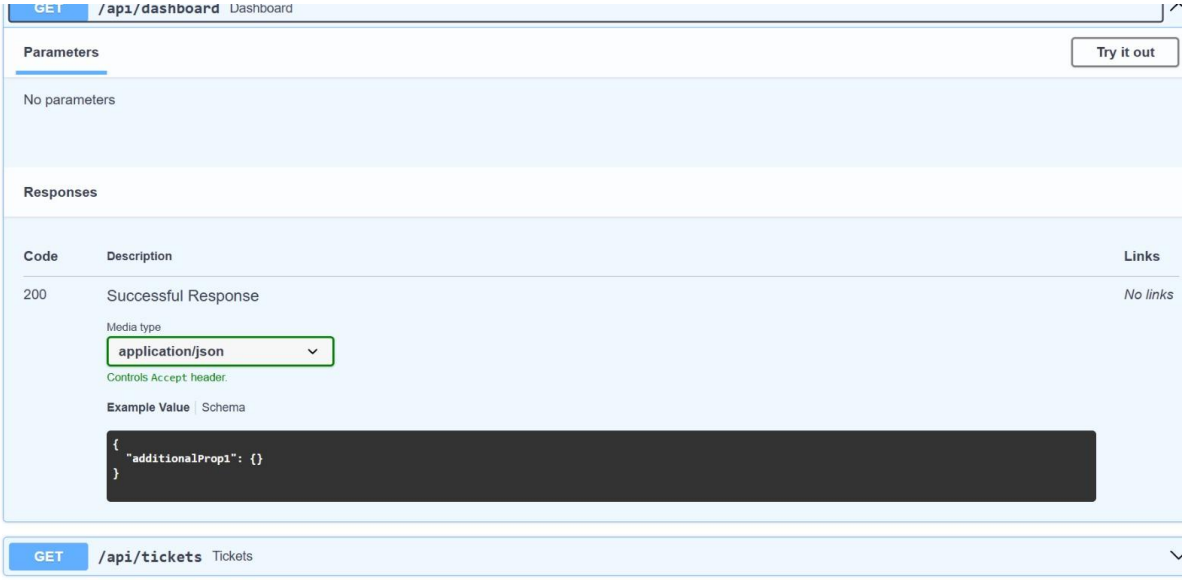
loc* > Expand all array<(string | integer)>

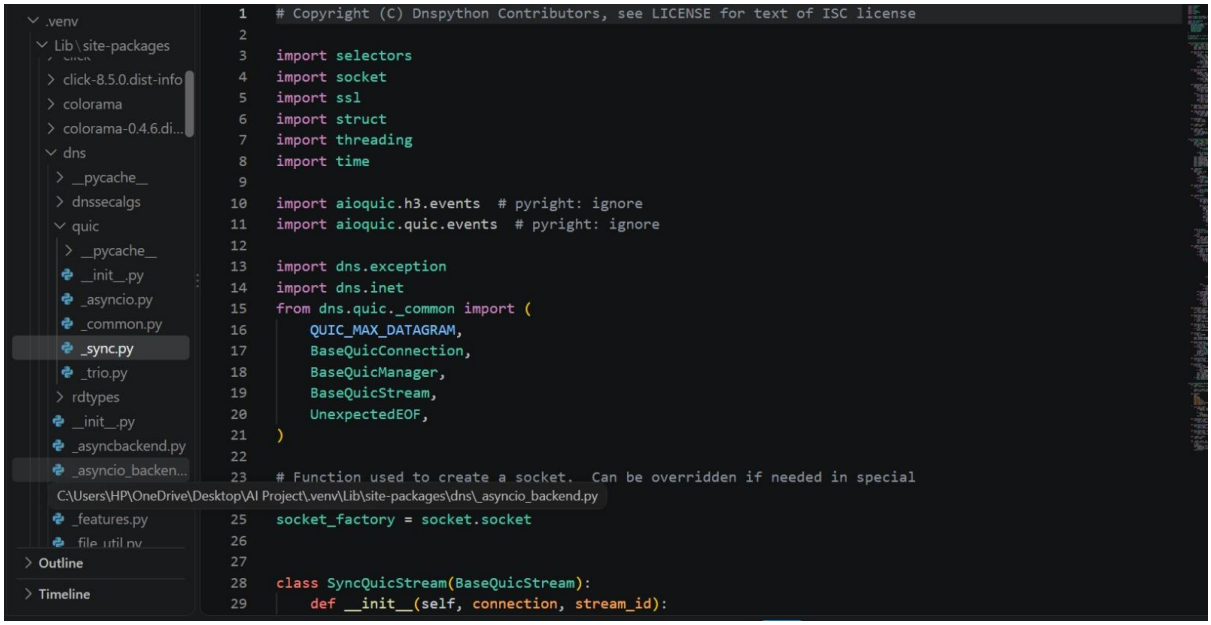
msg* string

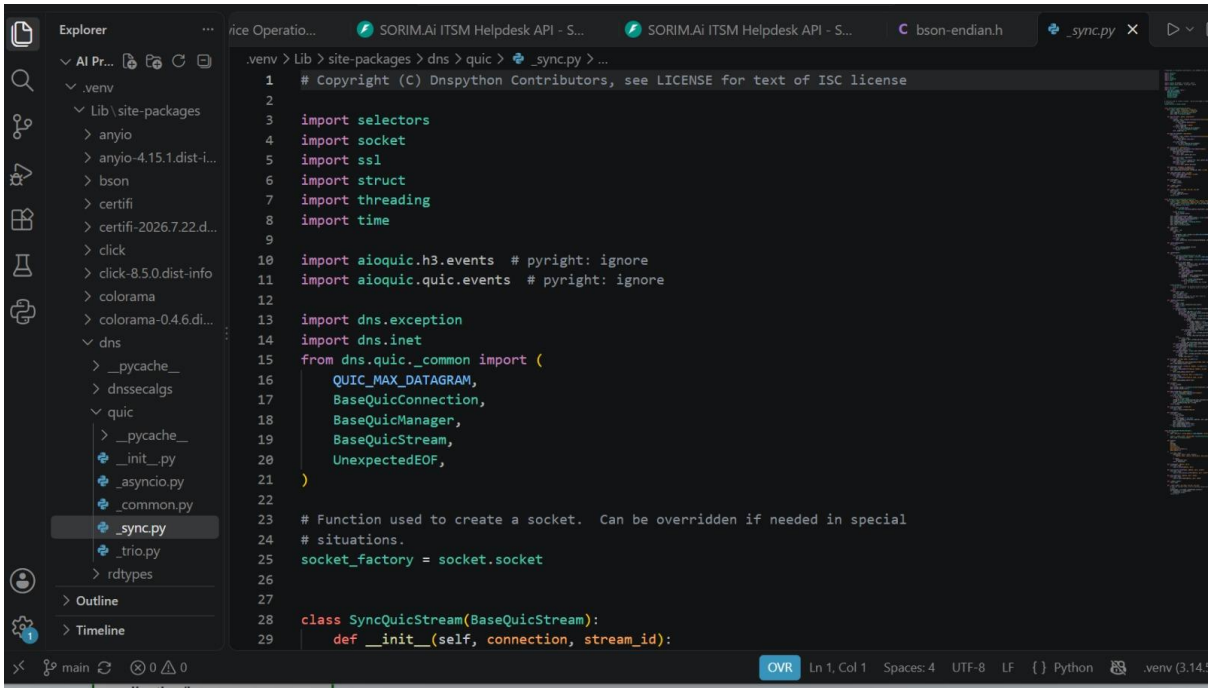
type* string

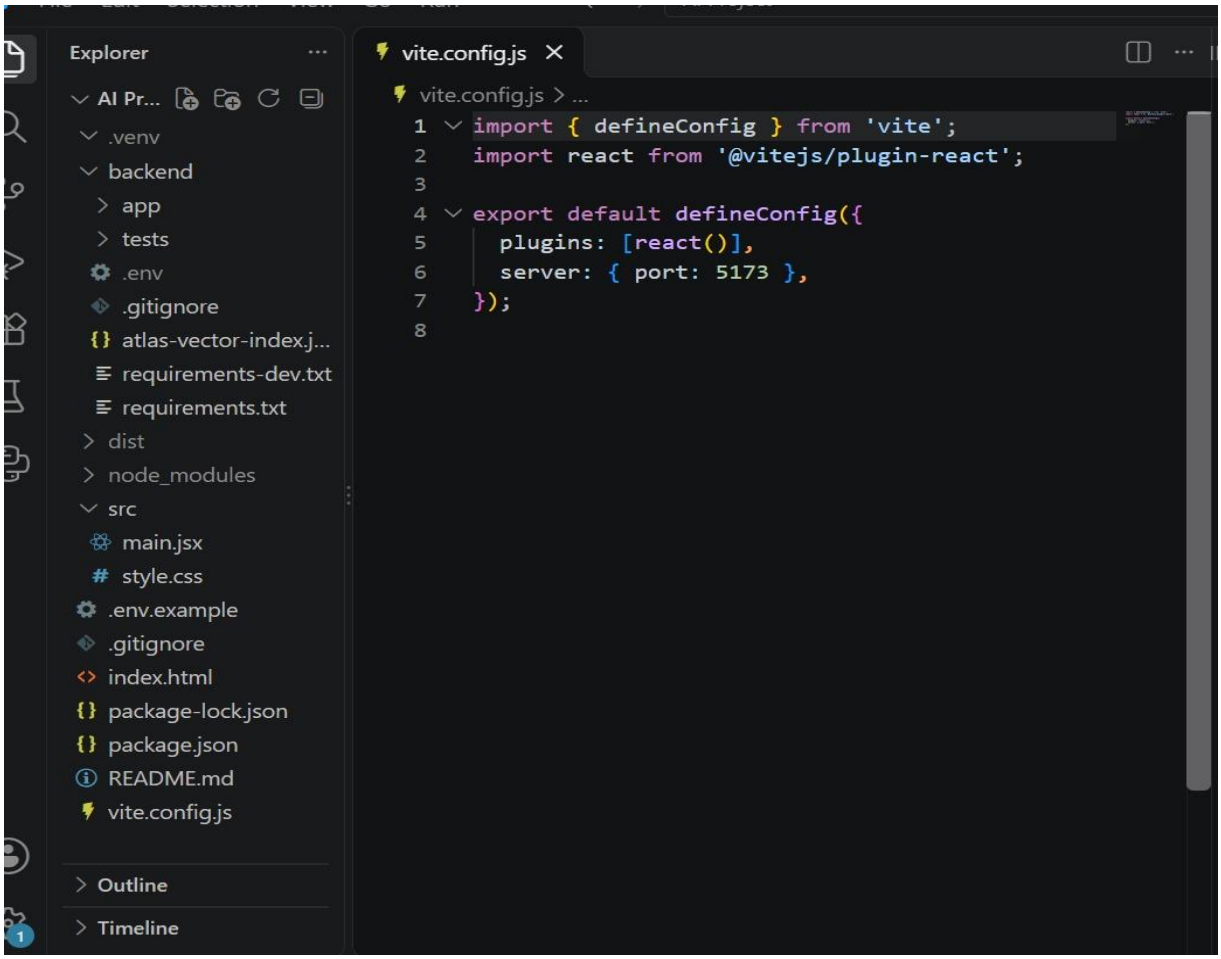
input any

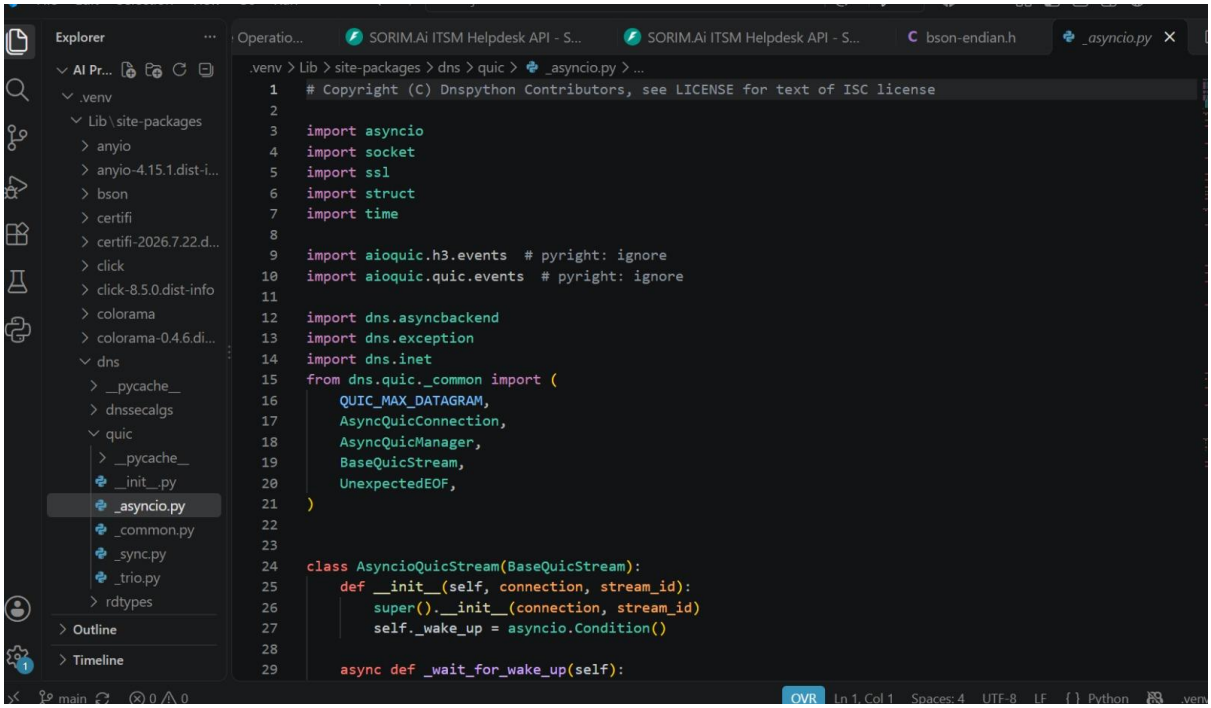
ctx object

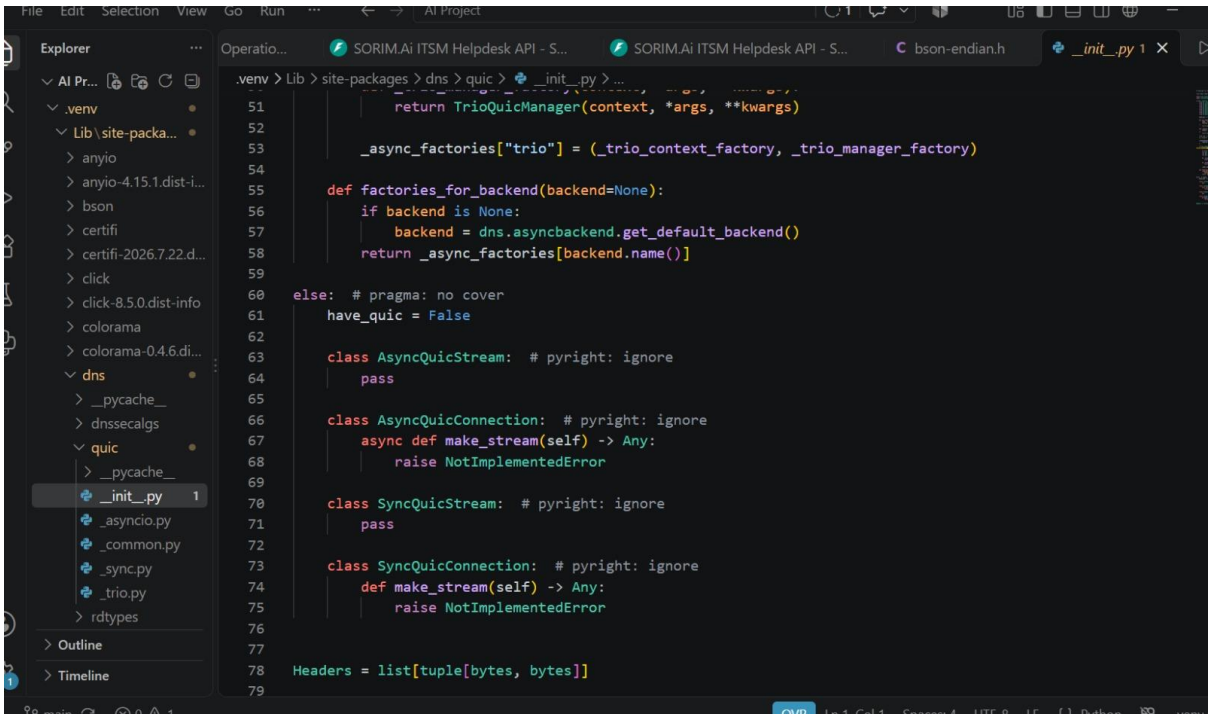

GET /api/dashboard Dashboard
Parameters
Try it out
No parameters
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"additionalProp1": {}
}
GET /api/tickets Tickets

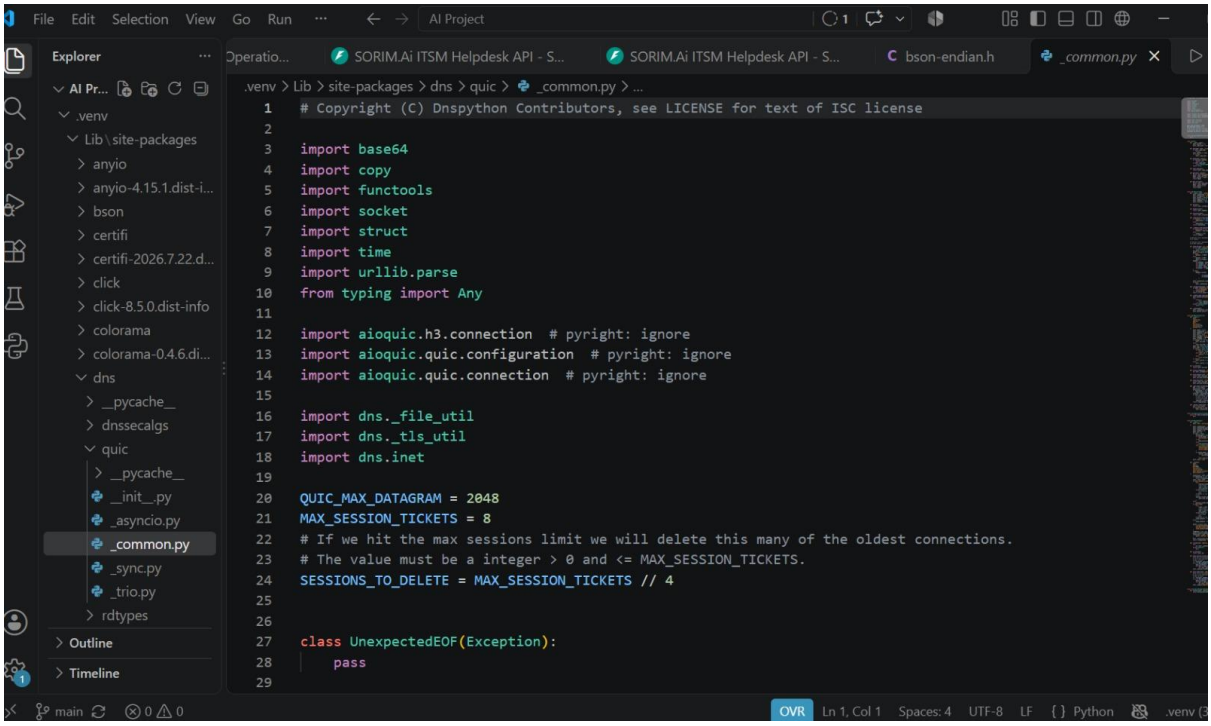

.venv
Lib\site-packages
click-8.5.0.dist-info
colorama
colorama-0.4.6.di...
dns
__pycache__
dnssecalgs
quic
__pycache__
__init__.py
_asyncio.py
_common.py
_sync.py
_trio.py
rdtypes
__init__.py
_asyncbackend.py
_asyncio_backen...
C:\Users\HP\OneDrive\Desktop\AI Project\.venv\Lib\site-packages\dns_asyncio_backend.py
_features.py
Outline
Timeline
Copyright (C) Dnspython Contributors, see LICENSE for text of ISC license
import selectors
import socket
import ssl
import struct
import threading
import time
import aioquic.h3.events # pyright: ignore
import aioquic.quic.events # pyright: ignore
import dns.exception
import dns.inet
from dns.quic._common import (
QUIC_MAX_DATAGRAM,
BaseQuicConnection,
BaseQuicManager,
BaseQuicStream,
UnexpectedEOF,
)
Function used to create a socket. Can be overridden if needed in special
socket_factory = socket.socket
class SyncQuicStream(BaseQuicStream):
def __init__(self, connection, stream_id):


Explorer
SORIM.Ai ITSM Helpdesk API - S...
SORIM.Ai ITSM Helpdesk API - S...
bson-endian.h
_sync.py
.venv > Lib > site-packages > dns > quic > _sync.py > ...
.venv
Lib \ site-packages
anyio
bson
certifi
click
colorama
dns
__pycache__
dnssecalgs
quic
__init__.py
_asyncio.py
_common.py
_sync.py
_trio.py
rdtypes
Outline
Timeline
Copyright (C) Dnspython Contributors, see LICENSE for text of ISC license
import selectors
import socket
import ssl
import struct
import threading
import time
import aioquic.h3.events # pyright: ignore
import aioquic.quic.events # pyright: ignore
import dns.exception
import dns.inet
from dns.quic._common import (
QUIC_MAX_DATAGRAM,
BaseQuicConnection,
BaseQuicManager,
BaseQuicStream,
UnexpectedEOF,
)
Function used to create a socket. Can be overridden if needed in special
situations.
socket_factory = socket.socket
class SyncQuicStream(BaseQuicStream):
def __init__(self, connection, stream_id):
main
OVR
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python


Explorer
AI Pr...
.venv
backend
app
tests
.env
.gitignore
atlas-vector-index.j...
requirements-dev.txt
requirements.txt
dist
node_modules
src
main.jsx
style.css
.env.example
.gitignore
index.html
package-lock.json
package.json
README.md
vite.config.js
Outline
Timeline
vite.config.js
vite.config.js > ...
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
plugins: [react()],
server: { port: 5173 },
});


Explorer
AI Pr...
.venv
Lib\site-packages
anyio
anyio-4.15.1.dist-i...
bson
certifi
certifi-2026.7.22.d...
click
click-8.5.0.dist-info
colorama
colorama-0.4.6.di...
dns
__pycache__
dnssecalgs
quic
__pycache__
__init__.py
_asyncio.py
_common.py
_sync.py
_trio.py
rdtypes
Outline
Timeline
Operatio...
SORIM.Ai ITSM Helpdesk API - S...
SORIM.Ai ITSM Helpdesk API - S...
bson-endian.h
_asyncio.py
.venv > Lib > site-packages > dns > quic > _asyncio.py > ...
Copyright (C) Dnspython Contributors, see LICENSE for text of ISC license
import asyncio
import socket
import ssl
import struct
import time
import aioquic.h3.events # pyright: ignore
import aioquic.quic.events # pyright: ignore
import dns.asyncbackend
import dns.exception
import dns.inet
from dns.quic._common import (
QUIC_MAX_DATAGRAM,
AsyncQuicConnection,
AsyncQuicManager,
BaseQuicStream,
UnexpectedEOF,
)
class AsyncioQuicStream(BaseQuicStream):
def __init__(self, connection, stream_id):
super().__init__(connection, stream_id)
self._wake_up = asyncio.Condition()
async def _wait_for_wake_up(self):
main
0 0
OVR
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python


File Edit Selection View Go Run
AI Project
Explorer
AI Pr...
.venv
Lib\site-packa...
anyio
anyio-4.15.1.dist-i...
bson
certifi
certifi-2026.7.22.d...
click
click-8.5.0.dist-info
colorama
colorama-0.4.6.di...
dns
__pycache__
dnssecalgs
quic
__pycache__
__init__.py 1
_asyncio.py
_common.py
_sync.py
_trio.py
rdtypes
Outline
Timeline
Operatio...
SORIM.Ai ITSM Helpdesk API - S...
SORIM.Ai ITSM Helpdesk API - S...
bson-endian.h
__init__.py 1
.venv > Lib > site-packages > dns > quic > __init__.py > ...
return TrioQuicManager(context, *args, **kwargs)
_async_factories["trio"] = (_trio_context_factory, _trio_manager_factory)
def factories_for_backend(backend=None):
if backend is None:
backend = dns.asyncbackend.get_default_backend()
return _async_factories[backend.name()]
else: # pragma: no cover
have_quic = False
class AsyncQuicStream: # pyright: ignore
pass
class AsyncQuicConnection: # pyright: ignore
async def make_stream(self) -> Any:
raise NotImplementedError
class SyncQuicStream: # pyright: ignore
pass
class SyncQuicConnection: # pyright: ignore
def make_stream(self) -> Any:
raise NotImplementedError
Headers = list[tuple[bytes, bytes]]




```python
# Copyright (C) Dnspython Contributors, see LICENSE for text of ISC license

from typing import Any

import dns._features
import dns.asyncbackend

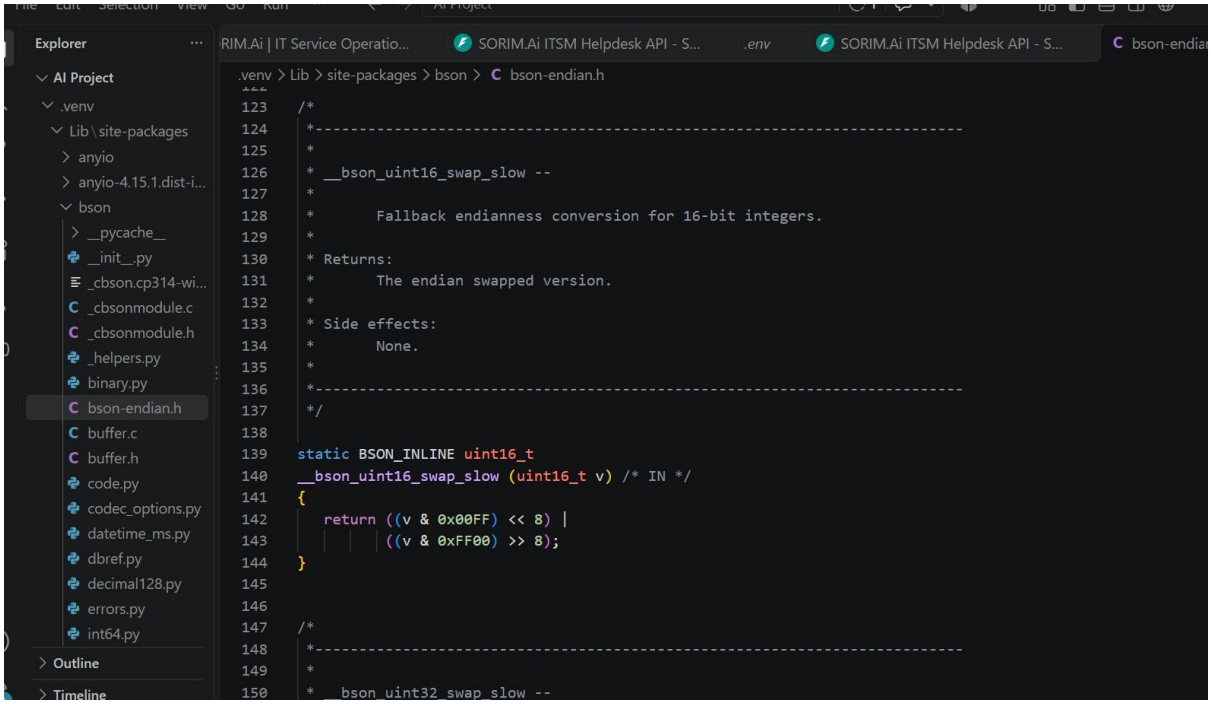
if dns._features.have("doq"):
    from dns._asyncbackend import NullContext
    from dns.quic._asyncio import AsyncioQuicConnection as AsyncioQuicConnection
    from dns.quic._asyncio import AsyncioQuicManager
    from dns.quic._asyncio import AsyncioQuicStream as AsyncioQuicStream
    from dns.quic._common import AsyncQuicConnection  # pyright: ignore
    from dns.quic._common import AsyncQuicManager as AsyncQuicManager
    from dns.quic._sync import SyncQuicConnection  # pyright: ignore
    from dns.quic._sync import SyncQuicStream  # pyright: ignore
    from dns.quic._sync import SyncQuicManager as SyncQuicManager

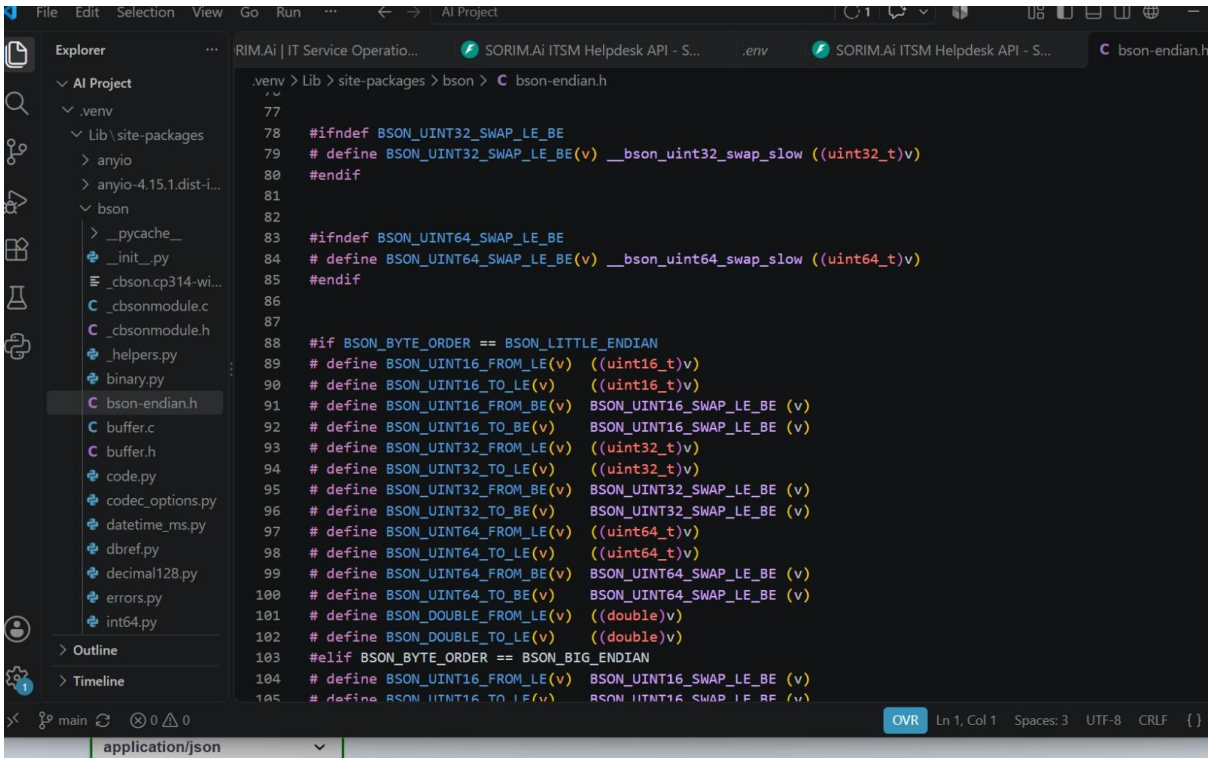
    have_quic = True

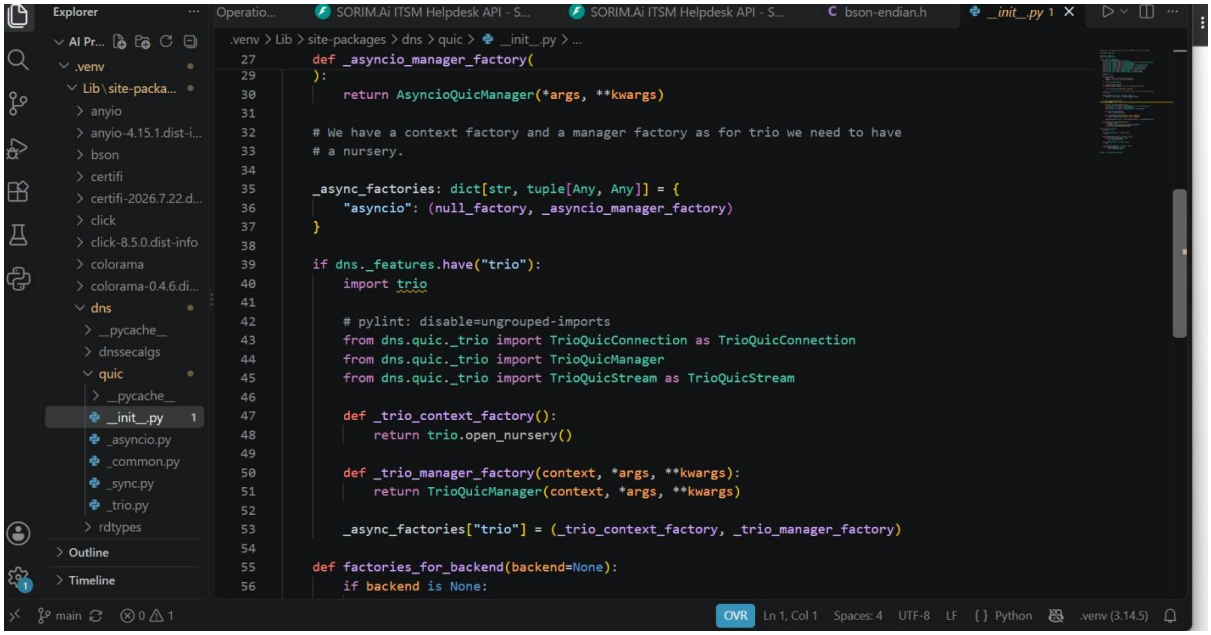
    def null_factory(
        *args,  # pylint: disable=unused-argument
        **kwargs,  # pylint: disable=unused-argument
    ):
        return NullContext(None)

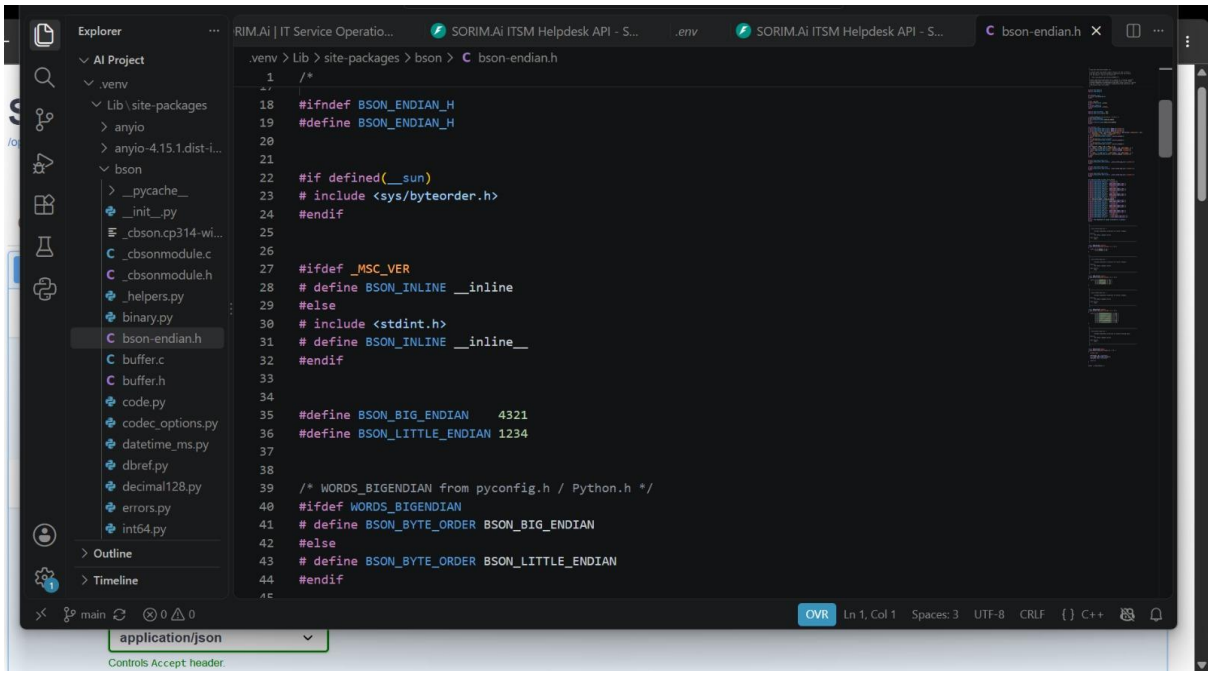
    def _asyncio_manager_factory(
        context, *args, **kwargs  # pylint: disable=unused-argument
    ):
```

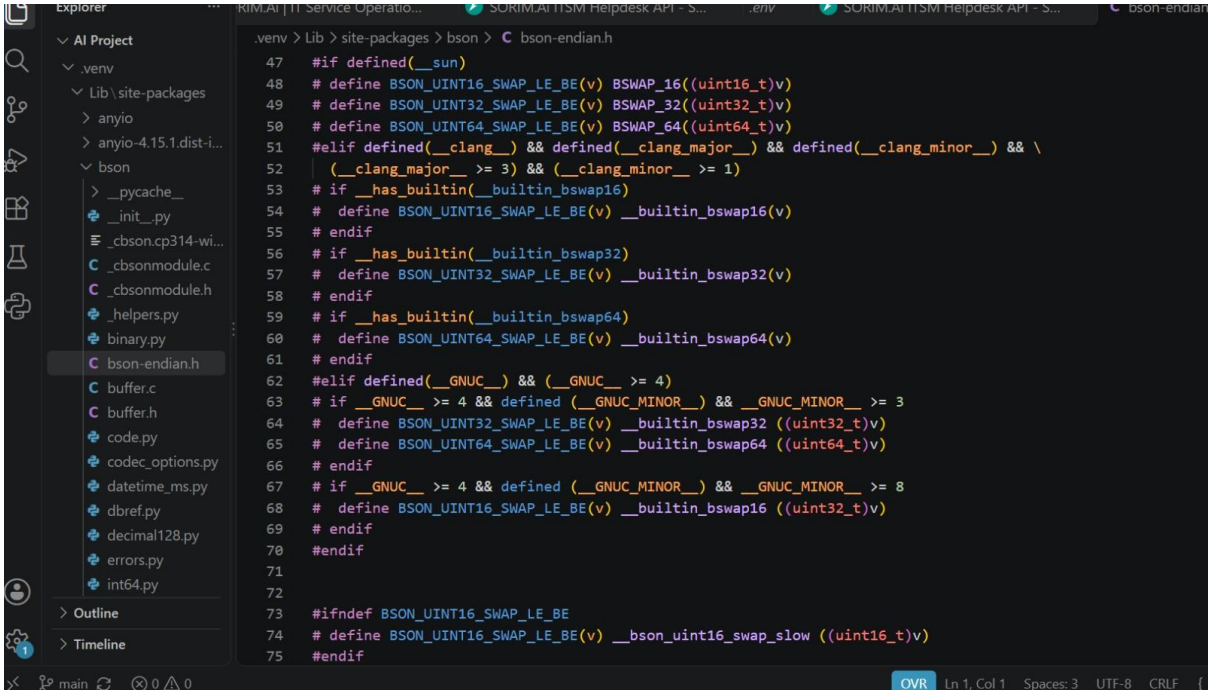
(module) _sync


Explorer
AI Project
.venv
Lib\site-packages
anyio
anyio-4.15.1.dist-i...
bson
__pycache__
__init__.py
_cbson.cp314-wi...
_cbsonmodule.c
_cbsonmodule.h
_helpers.py
binary.py
bson-endian.h
buffer.c
buffer.h
code.py
codec_options.py
datetime_ms.py
dbref.py
decimal128.py
errors.py
int64.py
Outline
Timeline
RIM.Ai | IT Service Operatio...
SORIM.Ai ITSM Helpdesk API - S...
.env
SORIM.Ai ITSM Helpdesk API - S...
bson-endian
.venv > Lib > site-packages > bson > C bson-endian.h
123 /*
124 *--
125 *
126 * __bson_uint16_swap_slow --
127 *
128 * Fallback endianness conversion for 16-bit integers.
129 *
130 * Returns:
131 * The endian swapped version.
132 *
133 * Side effects:
134 * None.
135 *
136 *--
137 */
138
139 static BSON_INLINE uint16_t
140 __bson_uint16_swap_slow (uint16_t v) /* IN */
141 {
142 return ((v & 0x00FF) << 8) |
143 ((v & 0xFF00) >> 8);
144 }
145
146
147 /*
148 *--
149 *
150 * __bson_uint32_swap_slow --


File Edit Selection View Go Run
AI Project
Explorer
AI Project
.venv
Lib\site-packages
anyio
anyio-4.15.1.dist-i...
bson
__pycache__
__init__.py
_cbson.cp314-wi...
_cbsonmodule.c
_cbsonmodule.h
_helpers.py
binary.py
bson-endian.h
buffer.c
buffer.h
code.py
codec_options.py
datetime_ms.py
dbref.py
decimal128.py
errors.py
int64.py
Outline
Timeline
RIM.Ai | IT Service Operatio...
SORIM.Ai ITSM Helpdesk API - S...
.env
SORIM.Ai ITSM Helpdesk API - S...
bson-endian.h
.venv > Lib > site-packages > bson > C bson-endian.h
77
78 #ifndef BSON_UINT32_SWAP_LE_BE
79 # define BSON_UINT32_SWAP_LE_BE(v) __bson_uint32_swap_slow ((uint32_t)v)
80 #endif
81
82
83 #ifndef BSON_UINT64_SWAP_LE_BE
84 # define BSON_UINT64_SWAP_LE_BE(v) __bson_uint64_swap_slow ((uint64_t)v)
85 #endif
86
87
88 #if BSON_BYTE_ORDER == BSON_LITTLE_ENDIAN
89 # define BSON_UINT16_FROM_LE(v) ((uint16_t)v)
90 # define BSON_UINT16_TO_LE(v) ((uint16_t)v)
91 # define BSON_UINT16_FROM_BE(v) BSON_UINT16_SWAP_LE_BE (v)
92 # define BSON_UINT16_TO_BE(v) BSON_UINT16_SWAP_LE_BE (v)
93 # define BSON_UINT32_FROM_LE(v) ((uint32_t)v)
94 # define BSON_UINT32_TO_LE(v) ((uint32_t)v)
95 # define BSON_UINT32_FROM_BE(v) BSON_UINT32_SWAP_LE_BE (v)
96 # define BSON_UINT32_TO_BE(v) BSON_UINT32_SWAP_LE_BE (v)
97 # define BSON_UINT64_FROM_LE(v) ((uint64_t)v)
98 # define BSON_UINT64_TO_LE(v) ((uint64_t)v)
99 # define BSON_UINT64_FROM_BE(v) BSON_UINT64_SWAP_LE_BE (v)
100 # define BSON_UINT64_TO_BE(v) BSON_UINT64_SWAP_LE_BE (v)
101 # define BSON_DOUBLE_FROM_LE(v) ((double)v)
102 # define BSON_DOUBLE_TO_LE(v) ((double)v)
103 #elif BSON_BYTE_ORDER == BSON_BIG_ENDIAN
104 # define BSON_UINT16_FROM_LE(v) BSON_UINT16_SWAP_LE_BE (v)
main
OVR
Ln 1, Col 1
Spaces: 3
UTF-8
CRLF
application/json


Explorer
AI Pr...
.venv
Lib \ site-packa...
anyio
anyio-4.15.1.dist-i...
bson
certifi
certifi-2026.7.22.d...
click
click-8.5.0.dist-info
colorama
colorama-0.4.6.di...
dns
__pycache__
dnssecalgs
quic
__pycache__
__init__.py 1
_asyncio.py
_common.py
_sync.py
_trio.py
rdtypes
Outline
Timeline
Operatio...
SORIM.Ai ITSM Helpdesk API - S...
SORIM.Ai ITSM Helpdesk API - S...
bson-endian.h
init.py 1
.venv > Lib > site-packages > dns > quic > __init__.py > ...
def _asyncio_manager_factory(
):
return AsyncioQuicManager(*args, **kwargs)
We have a context factory and a manager factory as for trio we need to have
a nursery.
_async_factories: dict[str, tuple[Any, Any]] = {
"asyncio": (null_factory, _asyncio_manager_factory)
}
if dns._features.have("trio"):
import trio
pylint: disable=ungrouped-imports
from dns.quic._trio import TrioQuicConnection as TrioQuicConnection
from dns.quic._trio import TrioQuicManager
from dns.quic._trio import TrioQuicStream as TrioQuicStream
def _trio_context_factory():
return trio.open_nursery()
def _trio_manager_factory(context, *args, **kwargs):
return TrioQuicManager(context, *args, **kwargs)
_async_factories["trio"] = (_trio_context_factory, _trio_manager_factory)
def factories_for_backend(backend=None):
if backend is None:
main
0 1
OVR
Ln 1, Col 1
Spaces: 4
UTF-8
LF
{ } Python
.venv (3.14.5)


Explorer
AI Project
.venv
Lib \ site-packages
anyio
anyio-4.15.1.dist-i...
bson
__pycache__
__init__.py
_cbson.cp314-wi...
_cbsonmodule.c
_cbsonmodule.h
_helpers.py
binary.py
bson-endian.h
buffer.c
buffer.h
code.py
codec_options.py
datetime_ms.py
dbref.py
decimal128.py
errors.py
int64.py
Outline
Timeline
RIM.Ai | IT Service Operatio...
SORIM.Ai ITSM Helpdesk API - S...
.env
SORIM.Ai ITSM Helpdesk API - S...
bson-endian.h
.venv > Lib > site-packages > bson > bson-endian.h
/*
#ifndef BSON_ENDIAN_H
#define BSON_ENDIAN_H
#if defined(__sun)
include <sys/byteorder.h>
#endif
#ifdef _MSC_VER
define BSON_INLINE __inline
#else
include <stdint.h>
define BSON_INLINE __inline__
#endif
#define BSON_BIG_ENDIAN 4321
#define BSON_LITTLE_ENDIAN 1234
/* WORDS_BIGENDIAN from pyconfig.h / Python.h */
#ifdef WORDS_BIGENDIAN
define BSON_BYTE_ORDER BSON_BIG_ENDIAN
#else
define BSON_BYTE_ORDER BSON_LITTLE_ENDIAN
#endif
main
0 0
OVR
Ln 1, Col 1
Spaces: 3
UTF-8
CRLF
{ } C++
application/json
Controls Accept header.


Explorer
AI Project
.venv
Lib \ site-packages
anyio
anyio-4.15.1.dist-i...
bson
__pycache__
__init__.py
_cbson.cp314-wi...
_cbsonmodule.c
_cbsonmodule.h
_helpers.py
binary.py
bson-endian.h
buffer.c
buffer.h
code.py
codec_options.py
datetime_ms.py
dbref.py
decimal128.py
errors.py
int64.py
Outline
Timeline
.venv > Lib > site-packages > bson > C bson-endian.h
47 #if defined(__sun)
48 # define BSON_UINT16_SWAP_LE_BE(v) BSWAP_16((uint16_t)v)
49 # define BSON_UINT32_SWAP_LE_BE(v) BSWAP_32((uint32_t)v)
50 # define BSON_UINT64_SWAP_LE_BE(v) BSWAP_64((uint64_t)v)
51 #elif defined(__clang__) && defined(__clang_major__) && defined(__clang_minor__) && \
52 (__clang_major__ >= 3) && (__clang_minor__ >= 1)
53 # if __has_builtin(__builtin_bswap16)
54 # define BSON_UINT16_SWAP_LE_BE(v) __builtin_bswap16(v)
55 # endif
56 # if __has_builtin(__builtin_bswap32)
57 # define BSON_UINT32_SWAP_LE_BE(v) __builtin_bswap32(v)
58 # endif
59 # if __has_builtin(__builtin_bswap64)
60 # define BSON_UINT64_SWAP_LE_BE(v) __builtin_bswap64(v)
61 # endif
62 #elif defined(__GNUC__) && (__GNUC__ >= 4)
63 # if __GNUC__ >= 4 && defined (__GNUC_MINOR__) && __GNUC_MINOR__ >= 3
64 # define BSON_UINT32_SWAP_LE_BE(v) __builtin_bswap32 ((uint32_t)v)
65 # define BSON_UINT64_SWAP_LE_BE(v) __builtin_bswap64 ((uint64_t)v)
66 # endif
67 # if __GNUC__ >= 4 && defined (__GNUC_MINOR__) && __GNUC_MINOR__ >= 8
68 # define BSON_UINT16_SWAP_LE_BE(v) __builtin_bswap16 ((uint32_t)v)
69 # endif
70 #endif
71
72
73 #ifndef BSON_UINT16_SWAP_LE_BE
74 # define BSON_UINT16_SWAP_LE_BE(v) __bson_uint16_swap_slow ((uint16_t)v)
75 #endif
main
0 0
OVR
Ln 1, Col 1
Spaces: 3
UTF-8
CRLF

AI-Powered ITSM Helpdesk & Intelligent Automation Platform

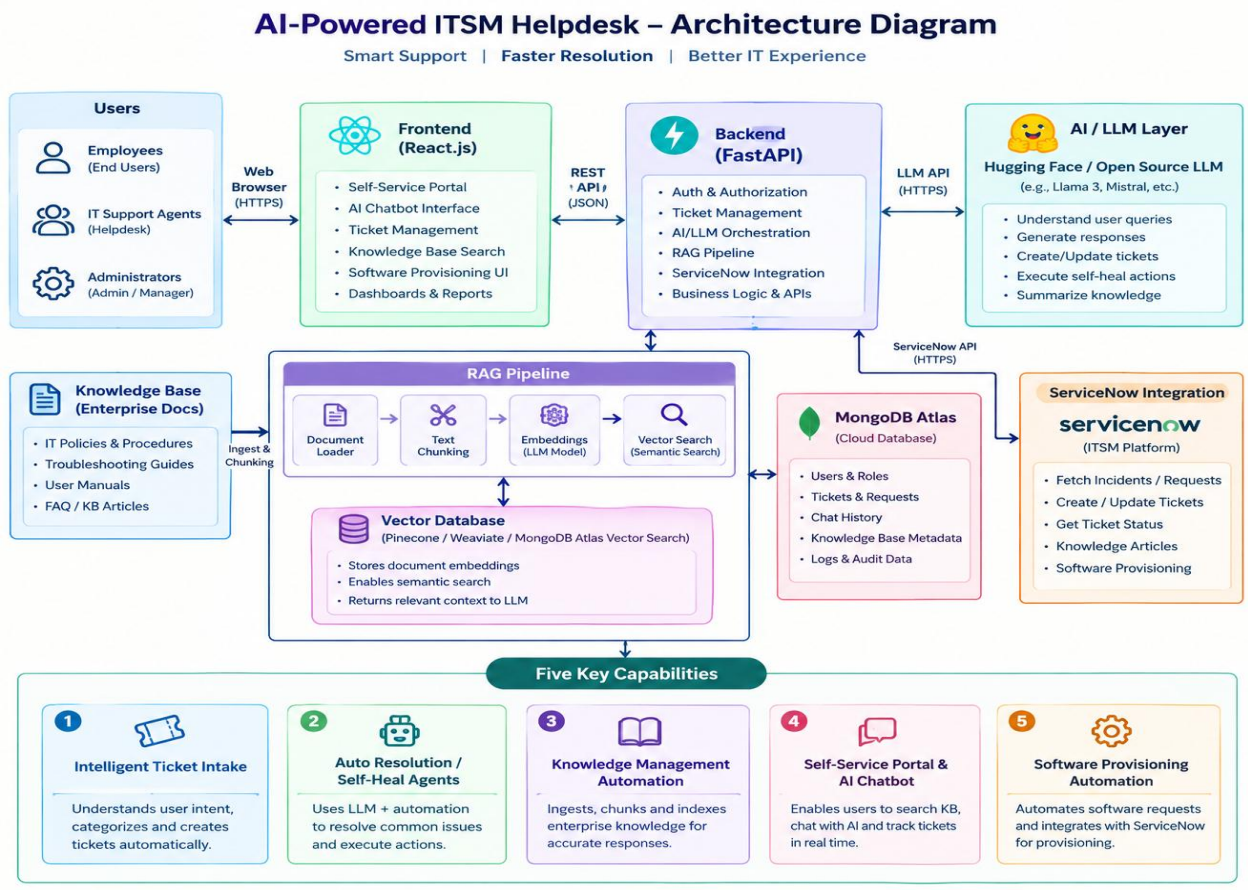
A prototype enterprise IT Service Management (ITSM) helpdesk that combines a React self-service portal, a FastAPI backend, retrieval-augmented generation (RAG), MongoDB Atlas, and ServiceNow (or a mock ServiceNow API) to classify employee requests, retrieve approved troubleshooting guidance, recommend resolutions, and create or update ITSM tickets.

Prototype note: This README documents a practical implementation plan for the assessment. Replace example values, route names, model identifiers, and configuration details with the actual values used in your repository. Do not claim a live ServiceNow connection unless it has been configured and tested.

1. Features and Assessment Coverage

Capability	What the prototype should demonstrate
Intelligent Ticket Intake	Detect intent, category/subcategory, priority, impact, urgency, assignment group, summary, and suggested resolution; create an incident through ServiceNow or a mock API.
Auto Resolution / Self-Heal Agent	Match safe, predefined issues (for example, password-expiry guidance or VPN status checks) to approved runbooks; validate the result, log the action, update the ticket, or escalate.
Knowledge Management Automation	Load 5–10 IT support documents, split them into chunks, generate embeddings, index them, retrieve relevant passages, and return answers with source attribution.
Self-Service Portal & AI Chatbot	Let employees ask questions, submit incidents/service requests, see responses and references, and escalate when the answer is uncertain.
Software Provisioning Automation	Route software requests through a catalogue and request workflow. Use a mock provisioning API for the demo; do not install software on a real device automatically.

2. Architecture



Main components

- **Frontend:** React.js with JavaScript or TypeScript. Provides the employee portal, chatbot, ticket submission, request status, ITSM dashboard, AI analysis, and knowledge base screens.
- **Backend:** Python and FastAPI. Exposes REST endpoints, validates input, orchestrates classification and RAG, applies safety rules, records audit events, and integrates with ServiceNow.
- **Database:** MongoDB Atlas stores application data such as ticket references, users, chat history, knowledge metadata, software requests, and audit logs. Keep secrets out of documents and logs.
- **LLM:** A Hugging Face-hosted or locally run open-source instruction model generates grounded responses and may assist with structured classification.
- **Embeddings:** A sentence-transformers model converts knowledge chunks and questions into vectors.

- **Vector store:** FAISS or Chroma supports similarity search. For a small local demo, FAISS is lightweight; Chroma is convenient when persisted collections and metadata filtering are useful. Choose one and document the actual choice.
- **ITSM:** ServiceNow REST API for incidents and service requests when credentials and a developer instance are available. Otherwise, use a clearly labelled mock API with the same interface.
- **Safe automation:** A controlled allowlist of demo actions. The LLM may suggest an action, but backend rules—not the LLM alone—decide whether it can run.

3. Suggested Technology Choices

Area	Suggested choice	Reason
Frontend	React + TypeScript (or JavaScript)	Component-based UI and clear request/response handling.
API	Python + FastAPI + Pydantic	Typed validation, automatic OpenAPI docs, and async-friendly endpoints.
Database	MongoDB Atlas	Managed document database for flexible ticket and conversation records.
LLM	HuggingFaceH4/zephyr-7b-beta or another available instruction-tuned open model	Demonstration of open-source instruction-following. Verify licence, hardware needs, availability, and provider limits before selecting.
Embeddings	sentence-transformers/all-MiniLM-L6-v2	Small, common sentence-embedding model suitable for a prototype.
Vector database	FAISS (local) or Chroma (persistent)	Simple semantic retrieval for a small knowledge collection.
ServiceNow	Table API / REST API	Create, read, and update incidents; use the instance's supported API and permissions.
Tests	Pytest + FastAPI TestClient; frontend test tooling	Validate core flows and failure cases.

Model choice guidance: Use an instruction-tuned model for answer generation and a separate embedding model for retrieval. Do not use the LLM as the vector database. Before finalising the model, check model-card instructions, licence, context length,

language support, inference latency, hardware requirements, and whether the chosen Hugging Face inference endpoint is available. If hosted inference is unavailable, configure a local model or return a controlled “AI service unavailable” response.

4. Repository Layout

A suggested layout is:

```
ai-powered-itsm-helpdesk/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/api.ts
│   │   └── App.tsx
│   ├── package.json
│   └── .env.example
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── routers/
│   │   │   ├── chat.py
│   │   │   ├── tickets.py
│   │   │   ├── knowledge.py
│   │   │   ├── automation.py
│   │   │   └── software.py
│   │   ├── services/
│   │   │   ├── classifier.py
│   │   │   ├── rag.py
│   │   │   ├── embeddings.py
```

```
│ │ │  ├── servicenow.py
│ │ │ └── automation.py
│ │ └── schemas/
│ ├── knowledge_docs/
│ ├── tests/
│ ├── requirements.txt
│ └── .env.example
├── sample_data/
├── README.md
└── .gitignore
```

Adapt this structure to match the actual files in your repository.

5. Prerequisites

Install or prepare:

- Python 3.10+ (use the version supported by your dependencies).
- Node.js LTS and npm.
- A MongoDB Atlas cluster and database user.
- A Hugging Face access token only if the selected model or inference provider requires one.
- Optional: ServiceNow developer instance with permission to access the relevant REST APIs.
- Git.

For local development, use separate terminals for the backend and frontend.

6. Setup Instructions

6.1 Clone the repository

git clone https://github.com/Janitha-I/AI-Powered-ITSM-Helpdesk.git

cd AI-Powered-ITSM-Helpdesk

If GitHub reports that the repository is not found, confirm with the owner that the repository still exists, that the URL is correct, and that you have access. A private

repository requires the owner to grant access. Do not assume the clone command will work until the repository is accessible.

6.2 Configure the backend

cd backend

python -m venv .venv

Activate the virtual environment:

Windows PowerShell

.venv\Scripts\Activate.ps1

Windows Command Prompt

.venv\Scripts\activate.bat

macOS / Linux

source .venv/bin/activate

Install dependencies:

pip install -r requirements.txt

Create backend/.env from .env.example and fill in your own values. Example:

APP_ENV=development

API_HOST=127.0.0.1

API_PORT=8000

FRONTEND_ORIGIN=http://localhost:5173

MONGODB_URI=mongodb+srv://<username>:<password>@<cluster-host>/<database>?retryWrites=true&w=majority

MONGODB_DATABASE=itsm_helpdesk

HUGGINGFACEHUB_API_TOKEN=

HF_MODEL_ID=HuggingFaceH4/zephyr-7b-beta

EMBEDDING_MODEL_ID=sentence-transformers/all-MiniLM-L6-v2

VECTOR_STORE=faiss

VECTOR_INDEX_PATH=./data/vector_index

KNOWLEDGE_DIR=./knowledge_docs

RAG_TOP_K=4

RAG_MIN_SCORE=0.35

SERVICENOW_BASE_URL=

SERVICENOW_USERNAME=

SERVICENOW_PASSWORD=

SERVICENOW_MODE=mock

These are example configuration names, not guaranteed to match the code. Rename them to match the application's settings. Keep SERVICENOW_MODE=mock until real integration has been tested. Never commit .env, real passwords, access tokens, database credentials, or API secrets.

Start the API (adjust the module path to your application):

uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

Open the FastAPI interactive documentation at http://127.0.0.1:8000/docs and the OpenAPI schema at http://127.0.0.1:8000/openapi.json.

6.3 Configure the frontend

Open a second terminal:

cd frontend

npm install

Create frontend/.env from .env.example. For a Vite-based React application:

VITE_API_BASE_URL=http://127.0.0.1:8000

Start the frontend:

npm run dev

Open the local URL printed by the development server (commonly http://localhost:5173). If your project uses Create React App or another tool, use its required environment variable prefix and start command instead.

6.4 Configure MongoDB Atlas

1. Create a database and database user in MongoDB Atlas.
2. Add your development IP address to Network Access; avoid opening access to all IPs unless there is a specific, temporary reason.
3. Copy the connection string and set the database name.
4. URL-encode special characters in the username/password where required.
5. Test the connection from the backend and handle connection failures clearly.

Suggested collections:

Collection	Purpose
tickets	Local ticket state and linked ServiceNow sys_id/number.
knowledge_documents	Document title, source, category, version, and metadata.
chat_sessions	Conversation/session metadata and messages, with appropriate retention controls.
software_requests	Requested software, requester, approval and mock provisioning status.
automation_audit_logs	Action name, actor, timestamp, result, and ticket/request reference.
users	Demo user profile and role data; avoid storing unnecessary personal data.

Do not store embedding vectors in MongoDB unless that is the deliberate design. The prototype may keep vectors in FAISS/Chroma and store document metadata in MongoDB.

7. Knowledge Base and RAG Approach

RAG (Retrieval-Augmented Generation) grounds AI answers in approved IT support material instead of relying only on the model's general knowledge.

Ingestion pipeline

1. Collect 5–10 approved sample documents, such as VPN Troubleshooting, Password Reset, Outlook Synchronization, Wi-Fi, Laptop Performance, Software Installation, and Application Access.
2. Extract text from each document and retain metadata: title, source, category, version, and last-updated date.

3. Clean repeated headers/footers and split text into overlapping chunks. A starting point is 400–700 tokens per chunk with 50–100 tokens of overlap; tune this against retrieval tests.
4. Generate an embedding for every chunk using the configured sentence-transformers model.
5. Store vectors and chunk metadata in FAISS or Chroma.
6. At question time, embed the query, retrieve the top-k relevant chunks, and optionally filter by category or document version.
7. Construct a prompt containing the retrieved passages and instruct the LLM to answer only from those passages.
8. Return the answer, source titles, and confidence/retrieval information to the UI.
9. If the evidence is missing or weak, say that the approved knowledge base does not contain enough information and offer escalation or ticket creation.

Retrieval and evaluation

- Start with top_k=4, then evaluate using a small set of realistic employee questions.
- Set a minimum relevance threshold based on the chosen vector store's score semantics; similarity scores differ across implementations, so do not copy a threshold blindly.
- Consider hybrid keyword + vector search for error codes, product names, and exact terms.
- Rebuild or update the index when source documents change.
- Test whether the returned sources actually support the generated answer.
- Never treat retrieved document text as trusted instructions to execute tools. Knowledge content is evidence, not permission to run actions.

Example grounded response

Question: “How do I troubleshoot VPN authentication failures?”

Expected answer: Provide only steps supported by the approved VPN guide, cite the guide's title, and offer to create an incident if the issue persists. Do not invent company-specific settings or credentials.

8. Model and AI Safety

- **Classification:** Ask the model for structured JSON fields such as intent, category, subcategory, priority, impact, urgency, assignment group, summary, and suggested resolution. Validate the output with Pydantic and enforce allowed values.
- **Generation:** Use the LLM to produce a concise response grounded in retrieved content.
- **Confidence:** Derive confidence from retrieval quality and validation signals; do not present an uncalibrated model self-rating as a reliable probability.
- **No hallucination:** If the approved knowledge base is insufficient, say so and escalate.
- **Source attribution:** Return the titles of the documents used for RAG.
- **Human escalation:** Low-confidence requests should offer ticket creation or hand-off to IT support.
- **Tool safety:** Use an allowlist of deterministic backend functions. Require validation and, where appropriate, human approval before any action that changes an account, device, or service.
- **Audit trail:** Log the requested action, validation result, execution result, and related ticket ID. Do not log passwords, tokens, or sensitive message content unnecessarily.
- **Prompt injection:** Treat user input and retrieved documents as untrusted data. The model must not override backend authorization or safety checks.

9. ServiceNow Integration

Modes

- **Mock mode:** Implement local endpoints with ServiceNow-like request/response shapes so the demo can run without a ServiceNow instance. Clearly mark records as mock data.
- **Live mode:** Configure a ServiceNow instance, API base URL, and least-privilege credentials. Use supported REST endpoints and verify required table fields and permissions for the instance.

Suggested flow

1. The classifier returns a validated intent and ticket fields.
2. The backend maps the fields to ServiceNow's incident/request schema.
3. The integration service calls the appropriate REST endpoint.

4. The backend stores the returned ticket number/sys_id and status in MongoDB.
5. The UI shows the result and any next steps.
6. Updates and automated actions are added to the audit log.

Typical incident operations use ServiceNow's Table API, subject to instance configuration and access control. A typical incident endpoint is /api/now/table/incident; verify the endpoint and field names against the target instance's current API documentation.

Field mapping example

Prototype field	Typical incident field (verify in instance)
Short summary	short_description
Full issue details	description
Priority	priority
Category	category
Subcategory	subcategory
Assignment group	assignment_group (often requires a valid sys_id)
Impact	impact
Urgency	urgency
Record ID	sys_id
Ticket number	number

ServiceNow may calculate or constrain priority based on impact and urgency, and choice values can vary by instance. Resolve assignment groups to valid IDs rather than sending display names blindly.

Required operations

- Create incident.
- Get incident.
- Update incident.
- Create a service request where practical and where the required Service Catalog configuration exists.

Use a timeout, safe error handling, and clear mock/live mode separation. Do not expose ServiceNow credentials to the browser. For a production deployment, prefer an approved OAuth or other enterprise authentication configuration instead of hard-coded credentials.

10. Suggested REST API Documentation

The following is a **proposed API contract** for the prototype. Match the route names and schemas to the implementation. FastAPI can publish the actual interactive API documentation at /docs.

Method	Endpoint	Purpose
GET	/health	Health and dependency status (avoid returning secrets).
POST	/api/chat	Ask a knowledge-grounded question; return answer, sources, and escalation option.
POST	/api/tickets/analyze	Classify a ticket description and return structured fields without creating a ticket.
POST	/api/tickets	Create an incident through ServiceNow or the mock adapter.
GET	/api/tickets/{ticket_id}	Retrieve local or linked ticket details.
PATCH	/api/tickets/{ticket_id}	Update permitted ticket fields/status.
GET	/api/knowledge	List knowledge documents and metadata.
POST	/api/knowledge/ingest	Ingest or re-index approved knowledge documents; protect this endpoint.
POST	/api/automation/resolve	Run an allowlisted, safe demo resolution flow.
POST	/api/software-requests	Submit a software request.
GET	/api/software-requests/{request_id}	Check software request/provisioning status.
GET	/api/dashboard/summary	Return ticket counts and automation status for the dashboard.

Example: analyse a ticket

Request

POST /api/tickets/analyze

Content-Type: application/json

{

"description": "I cannot connect to VPN since this morning. Authentication keeps failing.",

"requester_id": "demo-user-001"

}

Example response

{

"intent": "Incident",

"category": "network",

"subcategory": "vpn",

"priority": "P2",

"impact": "individual",

"urgency": "high",

"assignment_group": "Network Support",

"summary": "VPN authentication failure",

"suggested_resolution": "Retrieve the approved VPN troubleshooting steps.",

"sources": ["VPN Troubleshooting"],

"requires_escalation": false

}

This is illustrative output, not a guaranteed prediction. Validate priority and assignment using your business rules before creating a real incident.

Example: ask the RAG chatbot

Request

POST /api/chat

Content-Type: application/json

{

```
 "question": "How do I troubleshoot Outlook synchronization?",
 "session_id": "demo-session-001"
}
```

Example response

```
{
 "answer": "Follow the approved Outlook synchronization guide...",
 "sources": [
  {
   "title": "Outlook Synchronization",
   "chunk_id": "outlook-sync-01"
  }
 ],
 "confidence": "medium",
 "escalation_available": true
}
```

Example: create an incident

Request

```
POST /api/tickets
Content-Type: application/json
{
 "short_description": "VPN authentication failure",
 "description": "VPN authentication keeps failing.",
 "category": "network",
 "subcategory": "vpn",
 "priority": "2",
 "impact": "3",
 "urgency": "1"
}
```

Example response

```
{
 "ticket_id": "local-or-servicenow-id",
 "ticket_number": "INC0010123",
 "status": "created",
 "mode": "mock"
}
```

In mock mode, use a visibly mock ticket number and never imply that a real ServiceNow record was created.

Common HTTP status codes

Status	Meaning
200	Request completed successfully.
201	Record created.
400	Invalid request or unsupported operation.
401 / 403	Authentication or permission failure.
404	Ticket/document not found.
422	Request body failed schema validation.
429	Rate limit reached.
502 / 503	External AI or ServiceNow service unavailable.
500	Unexpected server error; return a safe message and log diagnostic details server-side.

11. Workflow Demonstrations

Scenario 1 — Intelligent ticket intake

Input: “I cannot connect to VPN since this morning. Authentication keeps failing.”

Expected flow:

1. React submits the request to FastAPI.

2. The classifier returns Incident, Network/VPN, impact, urgency, priority, summary, and suggested resolution.
3. RAG retrieves the approved VPN troubleshooting content.
4. The UI displays the suggestion and sources.
5. The employee can create an incident.
6. ServiceNow or the mock adapter returns a ticket reference.
7. The action is logged.

Scenario 2 — Self-heal workflow

Input: “My password has expired.”

Expected flow:

1. Detect a supported intent.
2. Retrieve the approved password-reset guide.
3. Offer the approved reset workflow or a mock action.
4. Validate the outcome where possible.
5. Record the action and result in the audit log.
6. Update or create a ticket if the issue remains unresolved.

Do not implement a real password reset unless a secure, authorised identity-management integration exists.

Scenario 3 — RAG

Input: “How do I troubleshoot Outlook synchronization?”

Expected flow: query embedding → vector retrieval → grounded answer → source titles → escalation option if retrieval is insufficient.

Scenario 4 — Software provisioning

Input: “I need Visual Studio Code installed on my laptop.”

Expected flow: detect Software Request → validate catalogue item → create request in ServiceNow or mock API → return request ID and status → show status in the UI. The prototype simulates provisioning; it does not install software on the employee's laptop.

Scenario 5 — Unsupported question

Input: a question not covered by the approved knowledge documents.

Expected flow: do not invent an answer → explain that the knowledge base does not contain enough information → offer ticket creation/escalation.

12. Testing

Create at least five test scenarios, including the five workflows above. Also test:

- Empty or malformed request body.
- LLM timeout or unavailable model.
- MongoDB unavailable.
- No relevant RAG documents.
- ServiceNow authentication failure or timeout.
- Invalid priority/category values.
- Duplicate submission or retry behaviour.
- Unsupported automation action.
- Software request with an unknown catalogue item.
- Secrets are not returned in API responses or written to logs.

Example backend test command:

pytest

Run frontend tests using the test script defined in frontend/package.json, if configured.

13. Security and Configuration

- Add .env, .venv/, node_modules/, generated vector indexes, cache files, and local logs to .gitignore.
- Commit only .env.example with placeholder values.
- Rotate any secret accidentally committed to Git.
- Validate every request with Pydantic or equivalent schema validation.
- Add authentication/role checks before exposing ticket, ingestion, dashboard, or automation routes.
- Restrict CORS to the intended frontend origin.
- Apply timeouts and rate limits to AI and external API calls.
- Use least-privilege database and ServiceNow credentials.

- Keep audit records for automated actions without storing unnecessary sensitive information.
- Require human approval for risky actions and make mock actions visibly identifiable.

14. Known Limitations

1. **Prototype, not production ITSM:** The application demonstrates workflows and is not a replacement for an organisation's fully governed ITSM deployment.
2. **Mock ServiceNow:** If no developer instance is configured, tickets and requests are local mock records and do not appear in a real ServiceNow instance.
3. **Model variability:** Open-source LLM responses can vary, be slow, or fail due to hardware, provider quotas, model availability, or network conditions.
4. **Classification is imperfect:** Intent, category, priority, and assignment group predictions need validation against business rules and test cases.
5. **RAG depends on source quality:** Answers may be incomplete if documents are missing, outdated, poorly parsed, or retrieved incorrectly.
6. **Vector search is approximate:** Similarity is not proof that a passage answers the question. Retrieval thresholds must be evaluated for the selected embedding and vector store.
7. **Demo automation is limited:** Self-healing and software provisioning should use safe mocks or approved sandbox integrations; the prototype does not change real endpoints by default.
8. **No guaranteed identity integration:** Authentication, SSO, role-based access, and employee directory integration require additional implementation unless already configured.
9. **No production monitoring by default:** Metrics, distributed tracing, alerting, backups, disaster recovery, and deployment hardening may not be included.
10. **Privacy and compliance:** Sample data should be synthetic. Real employee information requires approved retention, access-control, and compliance procedures.
11. **Rate limits and costs:** Hosted model and external API usage may be limited or incur costs.
12. **API examples are illustrative:** Confirm actual endpoint names, request fields, response schemas, and HTTP codes against the implemented code and target ServiceNow instance.